Report answers and evidence checklist

> **Screenshot filing made simple:** screenshots are grouped under `evidence/` by topic. The checklist is in EVIDENCE_GUIDE.md. The same images are included in this report's appendix.

Course: Data Engineering

Assignment: From Git to Gold

Student: Mahima Pandey

Roll number: 23BCS155

Repository: https://github.com/mahip004/freshcart-pipelines

This report uses the assignment brief, instructor-supplied FreshCart materials, and the student's screenshots. The submitted FreshCart notebook is sanitized: instructor credentials and outputs were removed, and the currency rule and sixth quality check were added. Snowflake and RabbitMQ were run in the student's own environments. The report includes the available GitHub, Snowflake, and RabbitMQ run evidence and explains the assignment scenarios. Mage will be added later.

A. Git and CI/CD

A1. Repository

The public repository is live at `https://github.com/mahip004/freshcart-pipelines`. It has separate commits for the project overview, CI, Mage blocks, FreshCart notebook, stream scripts, and report. `git log --oneline` output at report preparation time:

```
ceab90e Mark identity and repository checklist complete
1ef4134 Update submission status with verified CI runs
053933e Personalize report and record CI run evidence
93ed60e Restore the 80/20 split after the failing CI demonstration
adbcea7 Demonstrate failing CI with a 30 percent test split
a5658bd Fix pytest module import path in CI
113419e Document remaining account-run evidence
dd3952f Add numbered assignment answers and evidence checklist
a0c4256 Add Mage pipeline metadata at the CLI pipeline path
6f87031 Place the exporter in Mage's data_exporters directory
dfdfc4a Place the loader in Mage's data_loaders directory
1c6319e Add topic-matched drivers with ACK and NACK handling
037fb86 Add the topic exchange order dispatcher
b862ff4 Document the RabbitMQ run procedure and ACK behavior
1daf67a Add the sanitized FreshCart notebook with validation checks
00a605e Add FreshCart SQL validation and idempotency patterns
5fdd327 Add the Iris pipeline inspection notebook
f3be08c Wire the Mage pipeline and evaluation branch
9c279cd Add the predictions exporter block
4edec14 Add the evaluation branch with metrics
1a25367 Add the tested decision tree block
abc6c5a Add the reproducible train-test split block
b23b6cc Add the dataset validation block
fc0e2f3 Add the Iris data loader block
55a46ae Initialize Mage exporter package
da5c5bf Initialize Mage transformer package
c2e6e72 Initialize Mage loader package
8a774cf Initialize the Iris project package
518e96d Run pytest on pushes and pull requests
b1a949a Add the Iris split regression test
dd495df Declare Python project dependencies
e7fa66d Ignore secrets and generated artifacts
c230113 Add project overview and setup instructions
```

A2. Git command table

Command	Where the change happens	What exactly happens
`git init`	Working folder and local Git metadata	Creates a `.git` repository so Git can track this folder.
`git add train.py`	Staging area	Stages the current file contents for the next commit.
`git commit -m "msg"`	Local repository	Saves a snapshot of staged changes as a local commit.
`git remote add origin URL`	Local Git configuration	Records the remote URL under the name `origin`.
`git push -u origin main`	Remote Git	Uploads local `main` commits and sets upstream tracking.
`git pull origin main`	Local working folder and local Git	Fetches remote commits and integrates them into the current branch.

After `git commit`, a teammate cannot see the code on GitHub yet: the commit exists only in the local repository until pushed to a remote.

A3. CI

`tests/test_prepare_data.py` checks the required 120/30 split and the workflow runs pytest on pushes and pull requests using Python 3.12. The green run passed after the import-path fix: Actions run. The temporary `test_size=0.3` commit failed as expected with 105 training rows instead of 120: intentional failing run. Restoring `0.2` passed: fix run. The appendix includes the commit/check summary screenshot.

B. FreshCart ELT on Snowflake

The instructor notebook supplies the DDL, mock API fixtures and transformations. The target values from the assignment are compared with the captured baseline outputs below.

B1. Baseline

Check	Expected result from brief	Actual run
Bronze raw orders	8 per extraction	16 in the final table after two append-only extractions
Silver clean	6	6
Quarantine	2: ORD-1004, ORD-1006	2
Deduplicated orders	4	4
Gold aggregate	4 orders; INR 7,785.00; 126 points	4 orders; INR 7,785.00; 126 points
Rejected share	25%	25%

Captured result: one extraction contained 8 input records (6 web and 2 mobile). The final bronze-table screenshot shows 16 rows, consistent with two append-only extracts. Silver clean contains 6 rows, quarantine 2, deduplicated orders 4, and Gold 4 orders, INR 7,785.00 revenue, and 126 points. The run reports 25% rejected and zero for each displayed quality check.

Evidence: baseline summary, bronze source rows, silver rows, and quarantine/dedup results are in the Snowflake appendix. The separate connection screenshot is excluded because it displays account/user details.

B2. Record trace

ORD-1003's website bronze JSON uses flat keys such as `order_id`, `cust_email`, `cust_name`, `order_date`, `amount`, and `updated_at`; the mobile bronze JSON uses `orderId`, nested `customer.email` and `customer.name`, `orderDate`, nested `total.value` and `total.currency`, `deliveryPin`, and `updatedAt`. `SQL_10_ORDERS_CLEAN` maps these with `raw:order_id::STRING` etc. for web and `raw:orderId::STRING`, `raw:customer.email::STRING`, `raw:total.value::STRING` etc. for mobile. Both become normalized `order_id`, `email`, `name`, `order_date`, `amount`, `currency`, `pincode`, and `updated_at`; `LOWER(TRIM(...))`, `INITCAP`, `COALESCE(TRY_TO_DATE(...))`, and `TRY_TO_DECIMAL(REGEXP_REPLACE(...))` standardize them. `SQL_20_ORDERS_DEDUP` keeps the later mobile row using `QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC, ingest_seq DESC) = 1`. Meera's order is INR 2,100, so the FX rate is 1; Platinum's multiplier is 3, giving `FLOOR(2100 / 100) x 3 = 21 x 3 = 63` points.

ORD-1005's date is 2026-09-03, when Priya's SCD2 tier becomes Gold. The order-date range join to `dim.customers_scd2` selects Gold, then `ref.tier_multiplier` joins on `m.tier = c.tier`; points are `FLOOR(899 / 100) x 2 = 8 x 2 = 16`. ORD-1001 is dated 2026-09-01, so the same point-in-time SCD2 join selects Silver, yielding `FLOOR(1250 / 100) x 1 = 12` points.

B3. Currency validation

The submitted notebook adds `IFF(currency NOT IN ('INR', 'USD'), 'unsupported_currency', NULL)` to `ARRAY_CONSTRUCT_COMPACT(...)` in `SQL_10_ORDERS_CLEAN`. In the supplied mock data, ORD-1006 already has `bad_email` and `bad_pincode`; with the rule it also has `unsupported_currency` because its normalized currency is EUR. Its gold contribution remains zero because it was already excluded from `silver.orders_clean` and quarantined. The quarantine `INSERT` prevents another copy with the same source, null-safe order ID, and `updated_at` through its `NOT EXISTS` clause; a `QUALIFY ROW_NUMBER()` also retains one latest copy per source/order/version.

The added rule flags EUR as `unsupported_currency` while retaining the existing `bad_email` and `bad_pincode` reasons for ORD-1006. The quarantine table prevents duplicate source/order/version records, and the latest row per source/order/version is retained.

B4. SCD2 and idempotency

SCD2 outcome: CUST-02 has a Silver version from 2024-01-15. Adding Gold effective 2026-08-15 creates two versions and closes Silver on 2026-08-14. ORD-1002 is dated 2026-09-01, so the range join assigns Gold. Its INR value is `42.50 x 83.20 = INR 3,536`; loyalty points are `FLOOR(3536/100) x 2 = 70`. Gold points increase from 126 to 161. The changed order row has a new `row_hash`, so the `MERGE` updates `published_at` for ORD-1002.

Quantity	SCD2 scenario result
CUST-02 rows in `dim.customers_scd2`	2; Silver version closes 2026-08-14
Tier used for ORD-1002	Gold
ORD-1002 points	70 (INR 3,536 / 100 rounded down x Gold multiplier 2)
Total gold points	161 (126 + 35)
Gold rows with changed `published_at`	1: ORD-1002

`row_hash` gives each output row a fingerprint of relevant business values. The `MERGE` matches on a stable business key and updates a target only when the source hash differs, while inserting previously unseen keys. This prevents a rerun from duplicating unchanged gold facts and lets `published_at` remain stable when content is unchanged. Bronze still grows by eight rows per run because extraction appends a fresh copy of the raw API sample unless the landing/load step itself deduplicates it.

B5. Quality check and ELT

The notebook adds a sixth check for future orders: `SELECT COUNT_IF(order_date > CURRENT_DATE()) FROM silver.orders_dedup`. The baseline has zero future-dated orders. A valid order dated 2099-01-01 is identified as a problem; after its removal the check returns zero. ELT fits FreshCart because raw data can be loaded quickly into Snowflake and transformed there with SQL, keeping the original landing data available for reprocessing. Snowflake's warehouse can scale the transformations where the data already sits. ETL would win when transformations must happen before loading, such as when policy forbids transferring raw sensitive fields into the destination.

C. Mage Iris pipeline

C1. Graph and run - pending

Graph: `load_iris (data_loader) -> validate_iris (transformer) -> prepare_data (transformer) -> train_model (transformer) -> {save_results (data_exporter), evaluate_model (transformer)}`. Each block includes an attached `@test`. The project's metadata and importable blocks are included. **Run evidence required:** use Mage's CLI `mage run iris_project iris_classifier --test` in the configured Mage environment and append its output; The Mage Iris class materials and runtime were not supplied here.

C2. Test gate - pending

The `train_model` test enforces accuracy >= 0.85. A failing block test marks the block run as failed, so downstream exporters are not scheduled from that failed output; `save_results` therefore does not write a prediction file. This gate prevents invalid artifacts from being mistaken for valid outputs and avoids having to detect and remove a bad file later.

max_depth	Test accuracy	Test passed?	CSV created?
1	Pending Mage run	Pending	Pending
2	Pending Mage run	Pending	Pending
3	Pending Mage run	Pending	Pending
4	Pending Mage run	Pending	Pending
5	Pending Mage run	Pending	Pending

Before each run delete `iris_predictions.csv`. Save the Mage run output and file state when Mage is completed.

C3. Evaluation branch - pending

`metadata.yaml` includes the branch from `train_model` to `save_results` and `evaluate_model`. The block returns a confusion matrix and a classification metrics table. **Evidence required:** run the project in Mage, include the actual confusion matrix/metrics, and capture the Cloudflare-tunnel Mage UI graph. Mage Iris class materials and a live tunnel were not supplied.

C4. Own block - code prepared; Mage run pending

`validate_iris` checks for no missing values and exactly three classes, with its own `@test`; it sits after `load_iris`. Commit it and verify CI remains green.

D. Food-order stream

D1. Topic routing

#	Routing key	Matching queues
1	`bangalore.koramangala.food`	Ravi, Meena, Control
2	`bangalore.indiranagar.ride`	Priya, Arjun, Control
3	`bangalore.whitefield.ride`	Arjun, Control
4	`chennai.tnagar.food`	Karthik
5	`chennai.whitefield.ride`	dropped
6	`bangalore.koramangala`	Control only

These queue matches follow RabbitMQ topic-pattern rules. For key 6, # in `bangalore.#` matches zero or more words, so Control matches it. The driver patterns containing * require exactly one word after the zone or city and all city.zone.type patterns have three words; none match the two-word key.

D2. Build and run

`stream/dispatcher.py` publishes JSON every two seconds to the durable `rapido-dispatch` topic exchange. `stream/driver.py NAME PATTERN` creates a private exclusive queue, binds the topic, prints deliveries and ACKs valid messages. The screenshots show published routing keys and RabbitMQ UI activity with three temporary queues/consumers.

D3. ACK/NACK

A message without `zone` is rejected with `basic_nack(requeue=False)`; valid messages are ACKed. The dispatcher emits one malformed message every ten. If a driver crashes before ACK, the broker can redeliver the unacknowledged message after the connection closes. With `requeue=True`, a permanently malformed message may circulate continuously; rejecting without requeue or routing it to a dead-letter queue prevents a poison-message loop.

E. Integration and reflection

```
Git working tree -> commits -> GitHub remote -> Actions CI
                                  ■
                                  ■■■ tests Mage blocks before saving model output
                                  ■■■ FreshCart raw API -> bronze -> silver/quarantine -> gold in Snowflake
                                  ■■■ RabbitMQ topic stream -> driver queues -> (bonus) landing -> ELT -> gold
```

The four pieces form one delivery path: Git records pipeline code, CI checks changes, FreshCart transforms batch order data into gold tables, Mage trains and evaluates a tested classifier, and RabbitMQ routes live order events by topic. The optional landing consumer connects event batches to the same warehouse flow.

Reflection: The intentional CI failure was caused by temporarily changing the test split to 30%, which produced 105 training rows instead of the required 120. The failed GitHub check exposed the mismatch. Restoring the 80/20 split returned the expected 120 training rows and 30 test rows, and the follow-up Actions run passed.

Optional bonus: stream-to-lakehouse

The Snowflake and RabbitMQ services were run live for the main assignment. This optional bonus is a separate integration and is not claimed in this report version. To claim it, implement a consumer that batches RabbitMQ messages for one minute, writes them to a landing file, loads them into Snowflake, and reuses the FreshCart idempotent transformations and `MERGE`. Show the batch count and load result, then replay the same batch and demonstrate that the Gold order count, revenue, and points remain unchanged. A short consumer code excerpt plus run output and before/after Gold query results are sufficient evidence.

Final evidence checklist

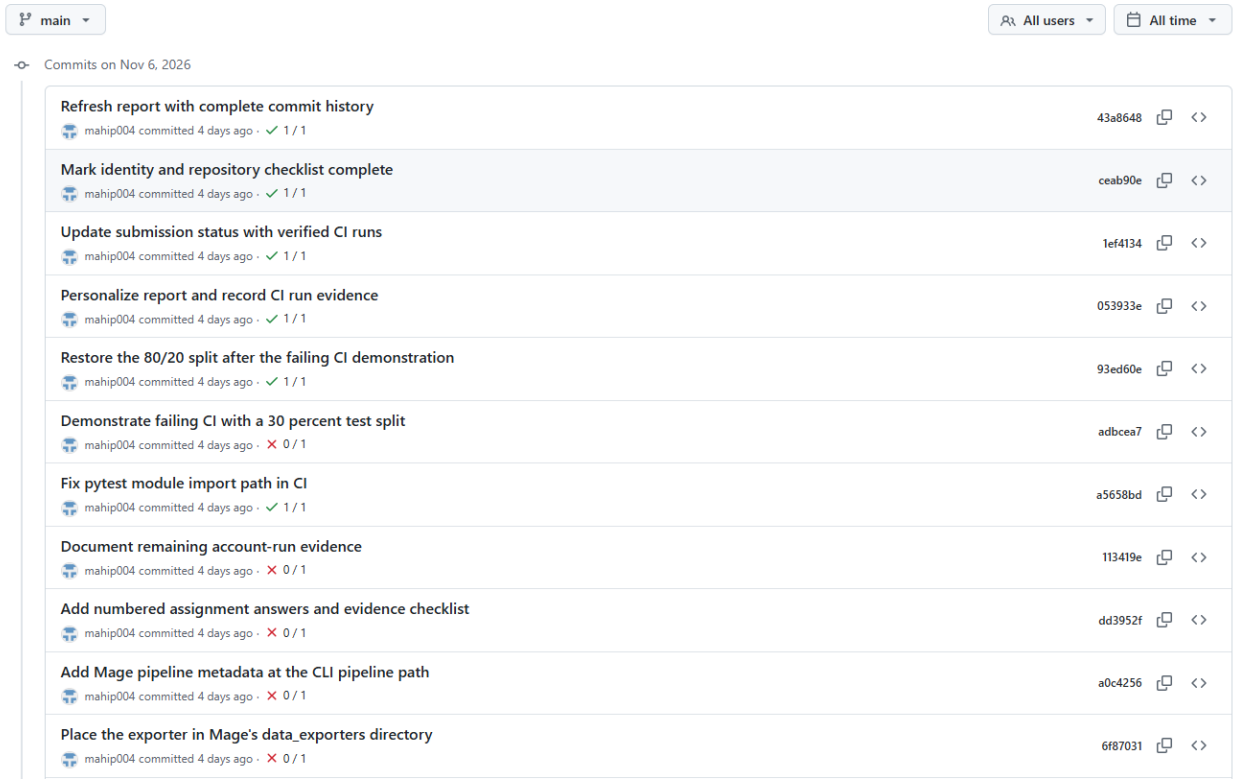
• [x] Add Mahima Pandey, roll number 23BCS155, and the GitHub repository URL to the cover.

• [x] Publish the repo with separate meaningful commits and include `git log --oneline`.

The non-Mage report sections are complete using the available screenshots, assignment materials, and code analysis. The appendix includes the supplied GitHub, Snowflake, and RabbitMQ screenshots. The intentional CI failure and passing fix provide the reflection example. The optional stream-to-lakehouse bonus is not claimed. Mage will be added when its screenshots are available.

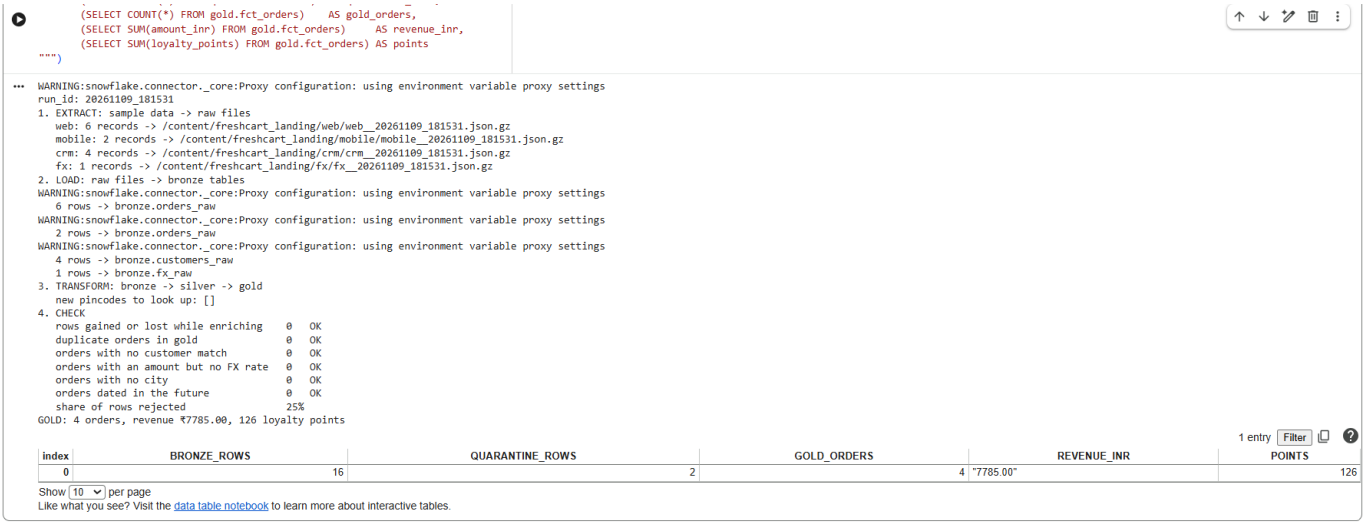
Evidence appendix - captured screenshots

GitHub



GitHub commit history with passing and failing checks

Snowflake



Snowflake baseline pipeline summary

9. Look inside each layer

Bronze: raw JSON exactly as received

```
# Inspect bronze records so we can verify the original source JSON was loaded with ingestion metadata.
show("SELECT ingest_seq, source, raw FROM bronze.orders_raw ORDER BY ingest_seq")
```

	INGEST_SEQ	SOURCE	RAW
0	1	web	{\n "amount": "1,250.00",\n "currency": "INR...
1	2	web	{\n "amount": null,\n "currency": "INR",\n ...
2	3	web	{\n "amount": "42.50",\n "currency": "USD",\...
3	4	web	{\n "amount": "-500",\n "currency": "INR",\n...
4	5	web	{\n "amount": "₹ 899",\n "currency": "inr",\...
5	6	web	{\n "amount": 30,\n "currency": "EUR",\n "c...
6	7	mobile	{\n "customer": {\n "email": "arjun.k@yaho...
7	8	mobile	{\n "customer": {\n "email": "Meera@Outloo...

Snowflake bronze raw rows

Silver: cleaned rows (6)

```
# Inspect cleaned orders so we can see standardized values and records that passed validation.
show("SELECT ingest_seq, source, order_id, email, name, order_date, amount, currency, pincode, amount_missing FROM silver.orders_clean ORDER BY ingest_seq")
```

	INGEST_SEQ	SOURCE	ORDER_ID	EMAIL	NAME	ORDER_DATE	AMOUNT	CURRENCY	PINCODE	AMOUNT_MISSING
0	1	web	ORD-1001	priya.r@gmail.com	Priya R	2026-09-01	1250.00	INR	600041	False
1	2	web	ORD-1003	meera@outlook.com	Meera S	2026-09-02	None	INR	400050	True
2	3	web	ORD-1002	arjun.k@yahoo.in	Arjun K	2026-09-01	42.50	USD	560034	False
3	5	web	ORD-1005	priya.r@gmail.com	Priya R	2026-09-03	899.00	INR	600041	False
4	7	mobile	ORD-1002	arjun.k@yahoo.in	Arjun K	2026-09-01	42.50	USD	560034	False
5	8	mobile	ORD-1003	meera@outlook.com	Meera S	2026-09-02	2100.00	INR	400050	False

Snowflake silver clean rows

Quarantine: rejected rows and why

```
# Inspect quarantined orders so we can understand which records failed validation and why.
show("SELECT run_id, source, order_id, reasons FROM quarantine.orders")
```

	RUN_ID	SOURCE	ORDER_ID	REASONS
0	20261109_181515	web	ORD-1004	[\n "non_positive_amount"\n]
1	20261109_181515	web	ORD-1006	[\n "bad_email",\n "bad_pincode",\n "unsupp...

Silver: one row per order (4)

```
# Inspect deduplicated orders so we can verify only the newest valid copy of each order remains.
show("SELECT order_id, ingest_seq, updated_at, amount FROM silver.orders_dedup ORDER BY order_id")
```

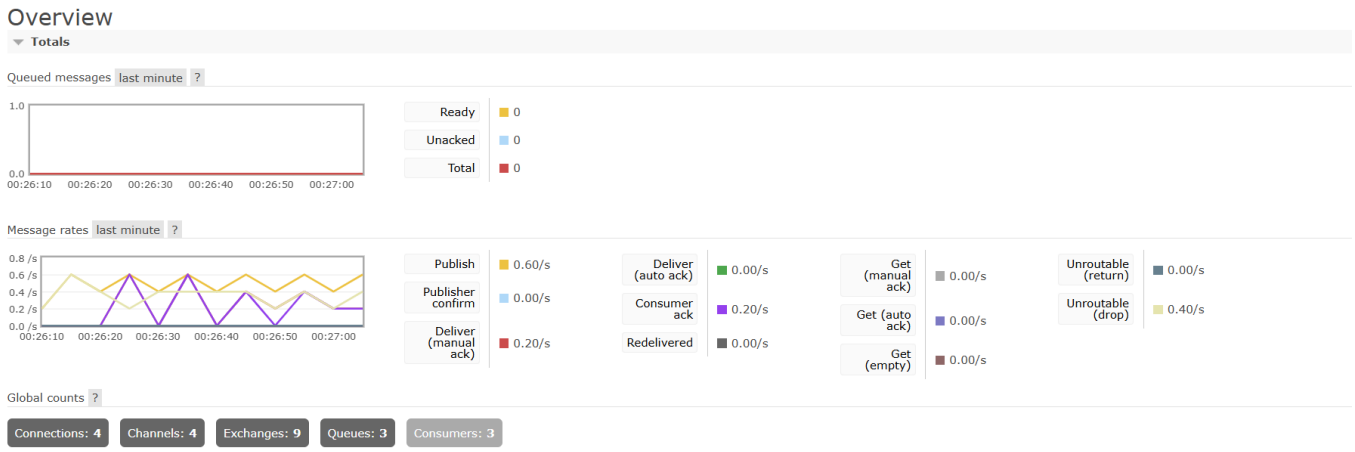
	ORDER_ID	INGEST_SEQ	UPDATED_AT	AMOUNT
0	ORD-1001	1	2026-09-01 10:02:00	1250.00
1	ORD-1002	7	2026-09-01 11:15:00	42.50
2	ORD-1003	8	2026-09-02 18:40:00	2100.00
3	ORD-1005	5	2026-09-03 16:45:00	899.00

Snowflake quarantine reasons and deduplicated rows

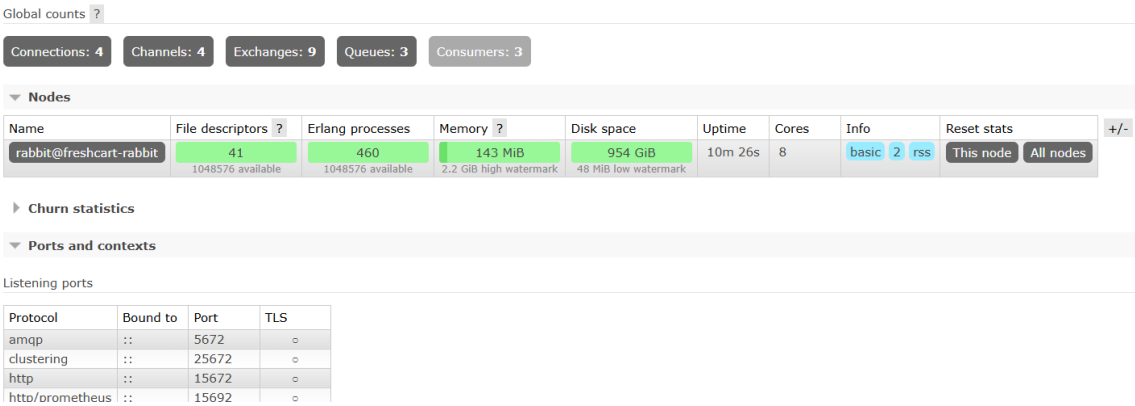
RabbitMQ

```
-09T18:56:12.298614+00:00'}
sent chennai.tnagar.ride: {'id': 'REQ-000003', 'city': 'chennai', 'zone': 'tnagar', 'type': 'ride', 'ts': '2026-11-09T18
:56:14.300142+00:00'}
sent chennai.velachery.ride: {'id': 'REQ-000004', 'city': 'chennai', 'zone': 'velachery', 'type': 'ride', 'ts': '2026-11
-09T18:56:16.301193+00:00'}
sent chennai.tnagar.ride: {'id': 'REQ-000005', 'city': 'chennai', 'zone': 'tnagar', 'type': 'ride', 'ts': '2026-11-09T18
:56:18.302512+00:00'}
sent chennai.tnagar.food: {'id': 'REQ-000006', 'city': 'chennai', 'zone': 'tnagar', 'type': 'food', 'ts': '2026-11-09T18
:56:20.303330+00:00'}
sent bangalore.indiranagar.ride: {'id': 'REQ-000007', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'ride', 'ts':
'2026-11-09T18:56:22.304240+00:00'}
sent chennai.velachery.food: {'id': 'REQ-000008', 'city': 'chennai', 'zone': 'velachery', 'type': 'food', 'ts': '2026-11
-09T18:56:24.305325+00:00'}
sent bangalore.indiranagar.food: {'id': 'REQ-000009', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'food', 'ts':
'2026-11-09T18:56:26.306753+00:00'}
sent chennai.missing.food: {'id': 'REQ-000010', 'city': 'chennai', 'type': 'food', 'ts': '2026-11-09T18:56:28.308118+00:
00'}
sent chennai.velachery.ride: {'id': 'REQ-000011', 'city': 'chennai', 'zone': 'velachery', 'type': 'ride', 'ts': '2026-11
-09T18:56:30.308618+00:00'}
sent chennai.tnagar.ride: {'id': 'REQ-000012', 'city': 'chennai', 'zone': 'tnagar', 'type': 'ride', 'ts': '2026-11-09T18
:56:32.309450+00:00'}
sent bangalore.koramangala.ride: {'id': 'REQ-000013', 'city': 'bangalore', 'zone': 'koramangala', 'type': 'ride', 'ts':
'2026-11-09T18:56:34.310912+00:00'}
sent chennai.tnagar.food: {'id': 'REQ-000014', 'city': 'chennai', 'zone': 'tnagar', 'type': 'food', 'ts': '2026-11-09T18
:56:36.312033+00:00'}
sent chennai.tnagar.food: {'id': 'REQ-000015', 'city': 'chennai', 'zone': 'tnagar', 'type': 'food', 'ts': '2026-11-09T18
:56:38.313427+00:00'}
sent chennai.velachery.ride: {'id': 'REQ-000016', 'city': 'chennai', 'zone': 'velachery', 'type': 'ride', 'ts': '2026-11
-09T18:56:40.314815+00:00'}
```

RabbitMQ dispatcher published events



RabbitMQ overview with active consumers and message rates



RabbitMQ temporary queues

Mage

Pending; no Mage screenshots or live-run results were supplied. This section will be completed after the Mage runs.